

Handling Date Values For Database Operations

05 September 2026 12:04

There are 2 date class available:

`java.util.Date` → represents both date and time

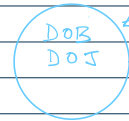
`java.sql.Date` → class ment for database operations

class represents only Date values
but not time.

`java.sql.Time`

`java.sql.Timestamp`

emp-id ← int
ename ← String
esalary ← double
eaddress ← String



String DOB = "28-05-1980";
↑
not recommended.

Handling Date values for Database operations

Sometimes as the part of programming requirement, we have to insert and retrieve Date like DOB, DOJ, DOP... with respect to database. It's not recommended to maintain date values in the form of String because comparison will become difficult.

In Java we have two date classes:

1. `java.util.Date`
2. `java.sql.Date`

`java.sql.Date` is specially designed class for handling Date values wrt database. Other than database operations; if we want to represent Date in our java program then we should go for `java.util.Date`.

`java.util.Date` can represent both date and time whereas `java.sql.Date` represents only Date but not time.

```
package MyJDBCOne;
import java.sql.Date;
public class TestDateAndTime {
    public static void main(String[] args) {
        java.util.Date ddate = new java.util.Date();
        long l = ddate.getTime();
        Date udate = new Date(l);
        System.out.println("java.util.Date = " + ddate);
        System.out.println("java.sql.Date = " + udate);
    }
}
```

Output:

```
java.util.Date = Sat Sep 05 12:30:25 IST 2026
java.sql.Date = 2026-09-05
```

Note: In SQL package, Time class is available to represent time values and Timestamp classes available to represent both date and time.

Inserting date values into Database:

Oracle: dd-mmm-yy

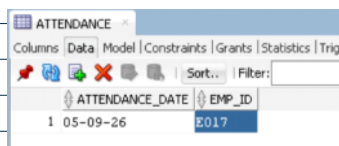
MySQL: yyyy-mm-dd

Statement: Java programmer is responsible to convert date values to respected DB date types.

```
PreparedStatement pst = ...
Java.util.Date attendanceDt = new java.util.Date();
long t = attendanceDt.getTime();
Java.sql.Date sqlDt = new java.sql.Date(t);
pst.setDate(1, sqlDt);
```

```
package MyJDBCOne;
import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;
public class DateInsertDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Connection con = ConnectionObject.getConnection();
        String sql = "INSERT INTO ATTENDANCE VALUES(?,?)";
        try {
            PreparedStatement pst = con.prepareStatement(sql);
            System.out.println("Enter employee id: ");
            String empid = sc.next();
            long l = new java.util.Date().getTime();
            Date sdate = new Date(l);
            pst.setDate(1, sdate);
            pst.setString(2, empid);
            pst.executeUpdate();
            System.out.println("Record inserted");
        } catch (SQLException e) {
            e.printStackTrace();
        }
        sc.close();
    }
}
```

Output:



ATTENDANCE_DATE	EMP_ID
05-09-26	E017

Retrieving Date type from database to Java program

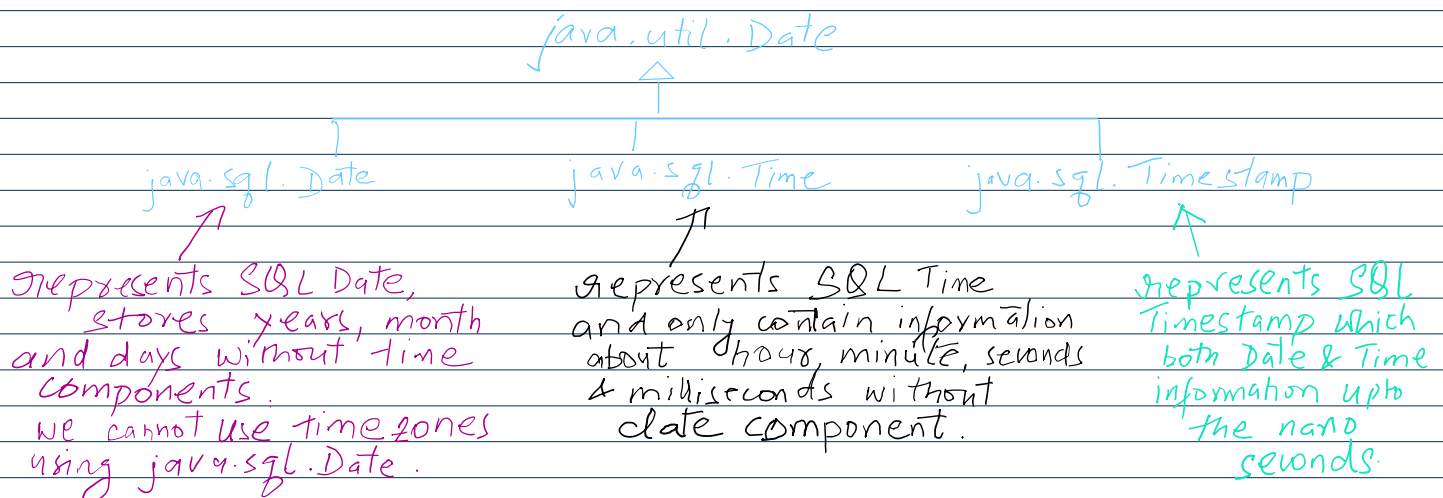
```
package MyJDBCOne;
import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.text.SimpleDateFormat;
public class DateRetrieveDemo {
    public static void main(String[] args) throws SQLException {
        Connection con = ConnectionObject.getConnection();
        String sql = "SELECT * FROM ATTENDANCE";
        PreparedStatement ps = con.prepareStatement(sql);
        ResultSet rs = ps.executeQuery();
        SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
        while (rs.next()) {
            Date adt = rs.getDate(1);
            String s = sdf.format(adt);
            String eid = rs.getString(2);
            System.out.println("Employee id: "+eid);
            System.out.println("Attendance date: " + s );
        }
        con.close();
    }
}
```

Output:

```
Employee id: E017
Attendance date: 05-09-2026
Employee id: E001
Attendance date: 06-09-2026
Employee id: E001
Attendance date: 06-09-20
```

java.sql.Date, java.sql.Time and java.sql.Timestamp

Three forms of date time fields which are date, time and timestamp. Each of these have a corresponding classes in JDBC and each of them extends java.util.Date.



Precision Database	SQL Date	SQL Time	SQL Timestamp
MySQL/Maria DB	Date/DateTime	Time	Timestamp
Oracle	Date		Timestamp, Timestamp with TimeZone, Timestamp with LocalTimeZone

Working with JDBC classes

Which class to choose is depends on the SQL type of the field. Prepared statement has setters for all three values setDate() for java.sql.Date, setTime() for java.sql.Time and setTimestamp() for java.sql.Timestamp.

```
package MyJDBCOne;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Date;

public class SQLDateTimeInsertExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/student_attendance";
        String username = "root";
        String password = "Calcutta1983";
        try (Connection con = DriverManager.getConnection(url, username,
password)){
            System.out.println("Database connected successfully");
            String SQL = "INSERT INTO test_datetime VALUES(?,?,?, ?, ?, ?)";
            PreparedStatement ps = con.prepareStatement(SQL);

            Date utilDate = new Date();
            java.sql.Date sqlDate = new java.sql.Date(utilDate.getTime());
            java.sql.Time sqlTime = new java.sql.Time(utilDate.getTime());
            java.sql.Timestamp sqlTS = new
java.sql.Timestamp(utilDate.getTime());
            ps.setDate(1, sqlDate);
            ps.setTime(2, sqlTime);
            ps.setTimestamp(3, sqlTS);

            { ps.setObject(4, utilDate);
              ps.setObject(5, utilDate);
              ps.setObject(6, utilDate); } ← without any problem it is
                                                    storing date, time, and
                                                    time stamp data in the table.
            int r = ps.executeUpdate();
            System.out.println("Number of rows inserted: "+ r);
        } catch (SQLException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
    }
}
```

Output:
Database connected successfully
Number of rows inserted: 1

	dtm_date	dtm_time	dtm_timestamp	obj_date	obj_time	obj_timestamp
▶	2026-09-09	18:45:39	2026-09-09 18:45:39	2026-09-09	18:45:39	2026-09-09 18:45:39